# TASK 5: Apply static Members and the Object Life Cycle

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Java Fundamentals and OOP - Assignment 1

## Static variables

The ticket counter is static: it belongs to the class, so every ticket in the program shares it and each new ticket gets the next number. An instance variable would be wrong, because each ParkingTicket would have its own counter, starting again at 1, and two tickets would have the same number.

```
class ParkingTicket {
    private static int counter = 0;     // one counter for the whole program
    private final int ticketNo;
    ParkingTicket(...) { this.ticketNo = ++counter; }
}
```

## Static methods and static blocks

A static method does not need an object. Creating a vehicle from its type is a good example: createVehicle(type, regNo) belongs to the facility's rules, not to one vehicle, so it is static. A static block runs once when the class is first loaded, and is suitable for setting up fixed data such as a table of rates. Our code uses a static final constant for the e-bike flat fee (FLAT_FEE = 20), since it is one value shared by all e-bikes.

## Life cycle of a ParkingTicket

A ticket is created when a vehicle enters and is stored in the slot array and in the active map. On exit, the code sets the slot to null and removes the ticket from the map. The ticket object then has no references pointing to it, so it becomes unreachable and eligible for garbage collection. The fee is calculated before this step, so nothing is lost.

## Passing and returning objects

Java passes object references by value. A method receives a copy of the reference, so it can change the object it points to, but cannot make the caller's variable point to a different object. In our code enter() returns the ticket, and exit() receives the registration number and the pass; the pass is an object, so the method reads its rule without changing it.

## Alternative: counter as an instance variable in two facilities

Suppose the university runs a north and a south facility. If the counter is an instance variable of ParkingFacility, each facility counts from 1 and the two will issue duplicate ticket numbers, which breaks the rule that a number is never reused. Two fixes exist: keep the counter static (one shared counter, so numbers are unique across both facilities, with a single int of space), or keep it per facility and add a facility prefix such as N-1001 and S-1001. The static version uses one int for any number k of facilities (O(1) space); the instance version uses k ints (O(k) space). Both use O(n) space for n tickets, because each ticket object stores its own number.